Chapter 9: Model Calibration

Youngseo Cho

2025-12-31

Contents

1 Introduction **1**

2 Section 9.1: Nonlinear Optimal Designs **1**
2.1 9.1.1 Analytical Models - SO2 Example 1
2.2 Model and Gradient Functions 2
2.3 D-optimal Design using ICAOD 2
2.4 9.1.2 Computationally Expensive Models - Functional Output 4

3 Section 9.2: Robust Designs **5**
3.1 9.2.1 Parameter Uncertainties 5
3.2 9.2.2 Model Uncertainties 7

4 Summary **9**

1 Introduction

This chapter covers optimal designs for model calibration including D-optimal designs for nonlinear models and robust designs.

Citation: V. R. Joseph (2026). *Experimental Design for Data Science and Engineering*. CRC Press.

```
library(numDeriv)
library(Matrix)
library(support)
library(MaxPro)
```

2 Section 9.1: Nonlinear Optimal Designs

2.1 9.1.1 Analytical Models - SO2 Example

```
# SO2 absorption model
so2 <- read.csv(file = "../datasets/so2.csv", header = TRUE)

# Nonlinear least squares fit
a <- nls(y1 ~ y0 + ((beta0 + beta1*x1)/(beta0 + beta1*x1 + x2) - y0) *
         (1 - exp(-(beta0 + beta1*x1 + x2)*dt/m)),
         start = list(beta0 = 0, beta1 = 1, m = 1000), data = so2)
summary(a)
```

```
##
```

```
## Formula: y1 ~ y0 + ((beta0 + beta1 * x1)/(beta0 + beta1 * x1 + x2) - y0) *
##     (1 - exp(-(beta0 + beta1 * x1 + x2) * dt/m))
##
## Parameters:
##        Estimate Std. Error t value Pr(>|t|)
## beta0  -1.53362    1.05597  -1.452    0.158
## beta1   0.91241    0.06771  13.476 1.67e-13 ***
## m     758.80650  109.57865   6.925 1.93e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.733e-05 on 27 degrees of freedom
##
## Number of iterations to convergence: 4
## Achieved convergence tolerance: 5.556e-07
```

2.2 Model and Gradient Functions

```
# Model function
h <- function(u, eta) {
  x <- u[1:3]
  val <- u[4] + ((eta[1] + eta[2]*x[1]) / (eta[1] + eta[2]*x[1] + x[2]) - u[4]) *
         (1 - exp(-(eta[1] + eta[2]*x[1] + x[2]) * x[3] / eta[3]))
  return(val)
}

# Gradient function
gradient <- function(x, eta) {
  term1 <- ((eta[1] + eta[2]*x[1]) / (eta[1] + eta[2]*x[1] + x[2]) - x[4])
  term2 <- exp(-(eta[1] + eta[2]*x[1] + x[2]) * x[3] / eta[3])
  v1 <- term1*term2*x[3]/eta[3] + x[2]/(eta[1] + eta[2]*x[1] + x[2])^2 * (1 - term2)
  v2 <- term1*term2*x[1]*x[3]/eta[3] + x[1]*x[2]/(eta[1] + eta[2]*x[1] + x[2])^2 * (1 - term2)
  v3 <- -term1*term2*(eta[1] + eta[2]*x[1] + x[2])*x[3]/eta[3]^2
  return(c(v1, v2, v3))
}
```

2.3 D-optimal Design using ICAOD

```
library(ICAOD)

# Suppress ICAOD iteration plots
pdf(NULL)

# Two-factor exponential model
# f(x) = eta0 + exp(-eta1*x1) + exp(-eta2*x2)

set.seed(1)
m <- 4

# Locally D-optimal design
opdes <- locally(formula = ~eta0 + exp(-eta1*x1) + exp(-eta2*x2),
                 predvars = c("x1", "x2"), parvars = c("eta0", "eta1", "eta2"),
                 lx = c(0, 0), ux = c(1, 1), inipars = c(0, 2, 2),
```

```
                iter = 100, k = m, family = gaussian(),
                ICA.control = list("trace" = FALSE))

D <- matrix(as.numeric(opdes$design[2:(2*m+1)]), nrow = m, ncol = 2)
w <- as.numeric(opdes$design[(2*m+2):(2*m+1+m)])

# Pseudo-Bayesian D-optimal design
pr <- uniform(lower = c(-10, 1, 1), upper = c(10, 3, 3))
opdes_bayes <- bayes(formula = ~eta0 + exp(-eta1*x1) + exp(-eta2*x2),
                     predvars = c("x1", "x2"), parvars = c("eta0", "eta1", "eta2"),
                     lx = c(0, 0), ux = c(1, 1), prior = pr,
                     iter = 100, k = m, family = gaussian(),
                     ICA.control = list("trace" = FALSE))

D_bayes <- matrix(as.numeric(opdes_bayes$design[2:(2*m+1)]), nrow = m, ncol = 2)
w_bayes <- as.numeric(opdes_bayes$design[(2*m+2):(2*m+1+m)])

# Close null device
dev.off()
```

```
## pdf
##   2
```

```
cat("Locally D-optimal Design Points:\n")
```

```
## Locally D-optimal Design Points:
```

```
print(round(D, 2))
```

```
##      [,1] [,2]
## [1,] 0.50  0.5
## [2,] 0.51  0.0
## [3,] 0.00  0.5
## [4,] 0.00  0.0
```

```
cat("\nWeights:", round(w, 3), "\n")
```

```
##
## Weights: 0.249 0.249 0.253 0.249
```

```
cat("\nBayesian D-optimal Design Points:\n")
```

```
##
## Bayesian D-optimal Design Points:
```

```
print(round(D_bayes, 2))
```

```
##      [,1] [,2]
## [1,]  0.0 0.00
## [2,]  0.5 0.49
## [3,]  0.5 0.00
## [4,]  0.0 0.51
```

```
cat("\nWeights:", round(w_bayes, 3), "\n")
```

```
##
## Weights: 0.263 0.239 0.254 0.244
```

2.4 9.1.2 Computationally Expensive Models - Functional Output

```
library(rkriging)
library(FNN)

# Suppress iteration plots
pdf(NULL)

set.seed(1)

# Read VPI simulation results
MP <- read.csv(file = "../datasets/functional/results.csv", header = TRUE)
q <- 5
D_func <- MP[-51, 1:q]
colnames(D_func) <- NULL

# Transform to [0,1] scale
D_func[, 1] <- (log(D_func[, 1]) - log(1e-12)) / (log(1e-9) - log(1e-12))
D_func[, 2] <- (D_func[, 2] - 4e-3) / (5e-3 - 4e-3)
D_func[, 3] <- (D_func[, 3] - 5e-3) / (6e-3 - 5e-3)
D_func[, 4] <- (D_func[, 4] - 500) / (2500 - 500)
D_func[, 5] <- (log(D_func[, 5]) - log(1e-3)) / (log(1e1) - log(1e-3))

N_func <- dim(D_func)[1]
pdet <- c(seq(0, 5000, length.out = 2001), seq(5000, 120000, length.out = 4601)[-1])
M_func <- length(pdet)

# Read functional output files
Y <- matrix(0, nrow = N_func, ncol = M_func)
for (i in 1:N_func) {
  filename <- paste0("../datasets/functional/run", i, ".txt")
  Y[i, ] <- read.table(file = filename)$V1
}

# Resample to uniform time grid
tsamp <- seq(0, max(pdet), length = 1001)
Ysamp <- Y[, 1:1001]
for (i in 1:N_func) {
  Ysamp[i, ] <- knn.reg(train = cbind(pdet), test = cbind(tsamp), y = Y[i, ], k = 1)$pred
}

dev.off()
```

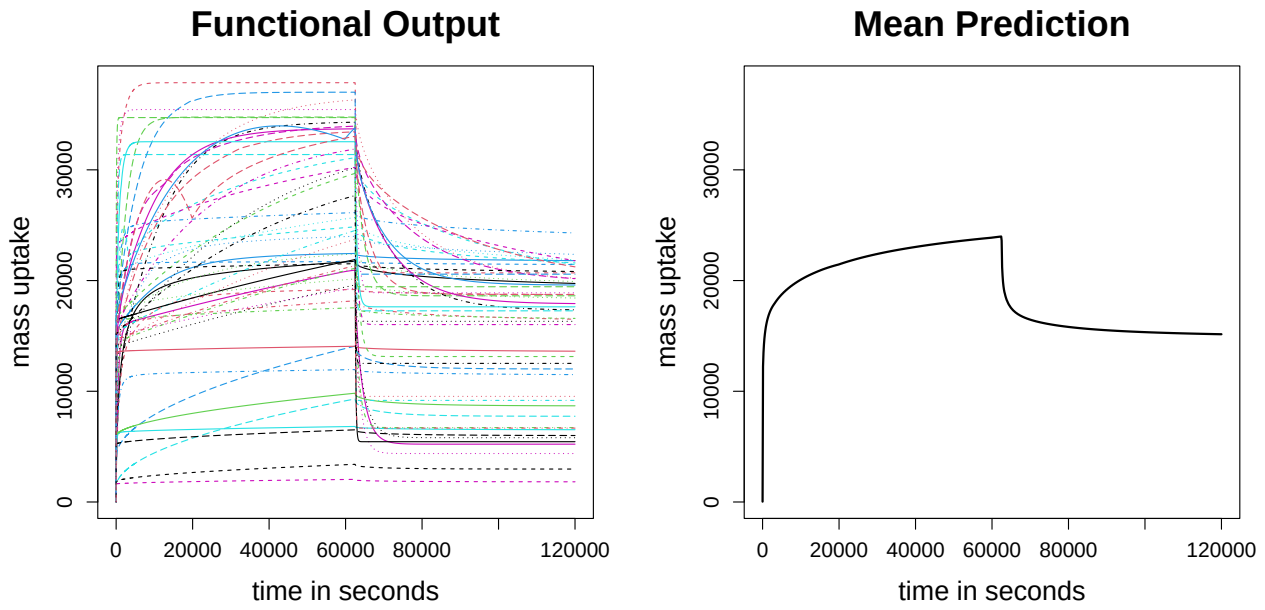
```
## pdf
##   2
```

```
# Figure 9.1: Functional output visualization
par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 2))

matplot(pdet, t(Y), "l", xlab = "time in seconds", ylab = "mass uptake",
        main = "Functional Output", cex.lab = 1.5, cex.main = 2, cex.axis = 1.2)

# Prediction at nominal parameter
y_mean <- apply(Ysamp, 2, mean)
```

```r
plot(tsamp, y_mean, "l", ylim = range(Y), xlab = "time in seconds", ylab = "mass uptake",
     main = "Mean Prediction", cex.lab = 1.5, cex.main = 2, cex.axis = 1.2, lwd = 2)
```




```r
par(mfrow = c(1, 1))
```

3 Section 9.2: Robust Designs

3.1 9.2.1 Parameter Uncertainties

```r
# Support points-based resampling
sp.sample <- function(x, n, prob, tol = 1e-6, iter.max = 10) {
  N <- nrow(x)
  x.dist <- as.matrix(dist(x))
  x.measure <- c(x.dist %*% prob)
  idx <- c(which.min(x.measure))
  if (prob[idx[1]] < 1/N) idx[1] <- which.max(prob)
  for (i in 2:n) {
    measure <- x.measure - apply(matrix(x.dist[, idx], ncol = (i-1)), 1, sum) / i
    idx <- c(idx, which.min(measure))
  }
  edist <- 2 * sum(x.measure[idx]) / n - sum(x.dist[idx, idx]) / n^2
  iter <- 0
  while (TRUE) {
    iter <- iter + 1
    for (i in 1:n) {
      measure <- x.measure - apply(x.dist[, idx[-i]], 1, sum) / n
      idx[i] <- which.min(measure)
    }
    edist.new <- 2 * sum(x.measure[idx]) / n - sum(x.dist[idx, idx]) / n^2
    if ((edist - edist.new) < tol || iter > iter.max) break
    edist <- edist.new
  }
  return(idx)
```

```
}

# Locally D-optimal vs Robust design
L <- matrix(c(0, 0, 1/2, 0, 0, 1/2, 1/2, 1/2), ncol = 2, byrow = TRUE)

set.seed(1)
N <- 20
p <- 2
samp <- sp(n = N, p = 2, dist.str = c("uniform", "normal"))$sp
```

```
## Optimizing ...
## [>                                                                    ] 0 %[>
## Maximum iterations reached: done!
```

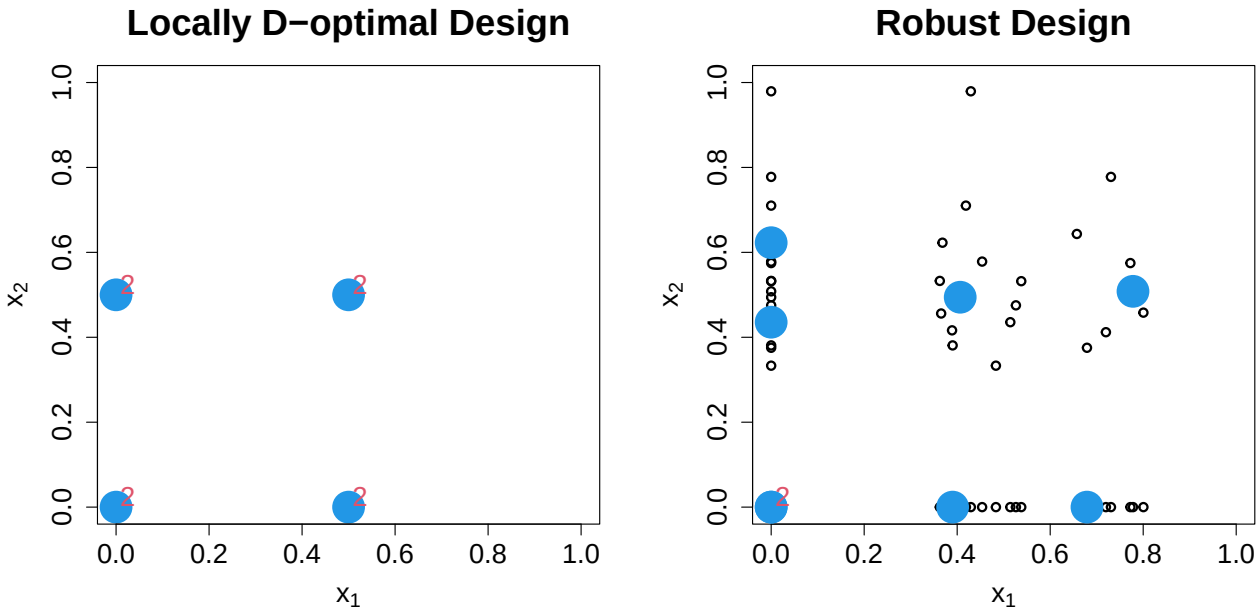
```
samp[, 1] <- 1 + 2 * samp[, 1]
samp[, 2] <- 2 + 1/2 * samp[, 2]

D <- NULL
for (i in 1:20) {
  D <- rbind(D, matrix(c(0, 0, 1/samp[i, 1], 0, 0, 1/samp[i, 2],
                         1/samp[i, 1], 1/samp[i, 2]), ncol = 2, byrow = TRUE))
}
ind <- sp.sample(D, 8, prob = rep(0.25/20, 80))

par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 2))

plot(L, pch = 16, col = 4, cex = 4, main = "Locally D-optimal Design",
     xlim = c(0, 1), ylim = c(0, 1), cex.main = 2,
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.lab = 1.5, cex.axis = 1.5)
text(L + 0.025, "2", col = 2, cex = 1.5)

plot(D, xlim = c(0, 1), ylim = c(0, 1), main = "Robust Design", cex.main = 2,
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.lab = 1.5, cex.axis = 1.5, lwd = 2)
points(D[ind, ], pch = 16, col = 4, cex = 4)
text(0.025, 0.025, "2", col = 2, cex = 1.5)
```




```
par(mfrow = c(1, 1))
```

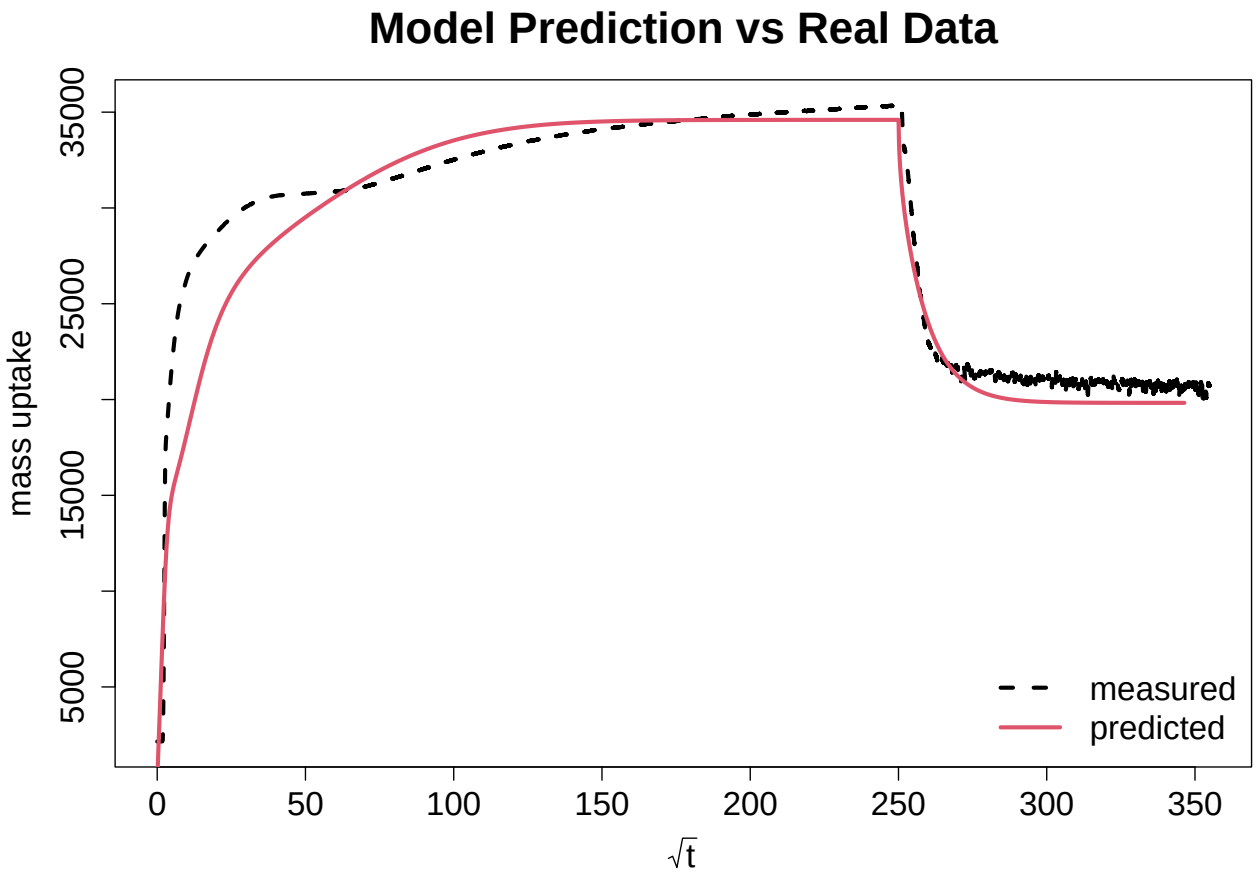
3.2 9.2.2 Model Uncertainties

3.2.1 Model Prediction vs Real Data

```
# Figure 9.3: Model prediction vs real experimental data
best <- read.table(file = "../datasets/functional/run51.txt")$V1
pdet <- c(seq(0, 5000, length.out = 2001), seq(5000, 120000, length.out = 4601)[-1])
sorp <- read.csv(file = "../datasets/functional/exp_sorption_130c_8.7torr_483nm.csv")
desorp <- read.csv(file = "../datasets/functional/exp_desorption_130c_8.7torr_483nm.csv")

correct <- 35358.89275 - 34644.0896 - 1
yexp <- c(sorp[, 2], desorp[, 2] + correct)
texp <- c(sorp[, 1]^2, desorp[, 1]^2 + 62973)

par(mar = c(5, 5, 4, 2))
plot(sqrt(texp), yexp, main = "Model Prediction vs Real Data", cex.main = 2,
     xlab = expression(sqrt(t)), ylab = "mass uptake", lwd = 3,
     cex.lab = 1.5, cex.axis = 1.5, type = "l", lty = 2)
lines(sqrt(pdet), best, col = 2, lwd = 3)
legend("bottomright", legend = c("measured", "predicted"),
       col = c(1, 2), lty = c(2, 1), bty = "n", cex = 1.5, lwd = c(3, 3))
```



3.2.2 Design Augmentation

```
set.seed(1)
N <- 20
p <- 2
samp <- sp(n = N, p = 2, dist.str = c("uniform", "normal"))$sp
```

```
## Optimizing ...
## [>                                                                  ] 0 %[>
## Maximum iterations reached: done!
```

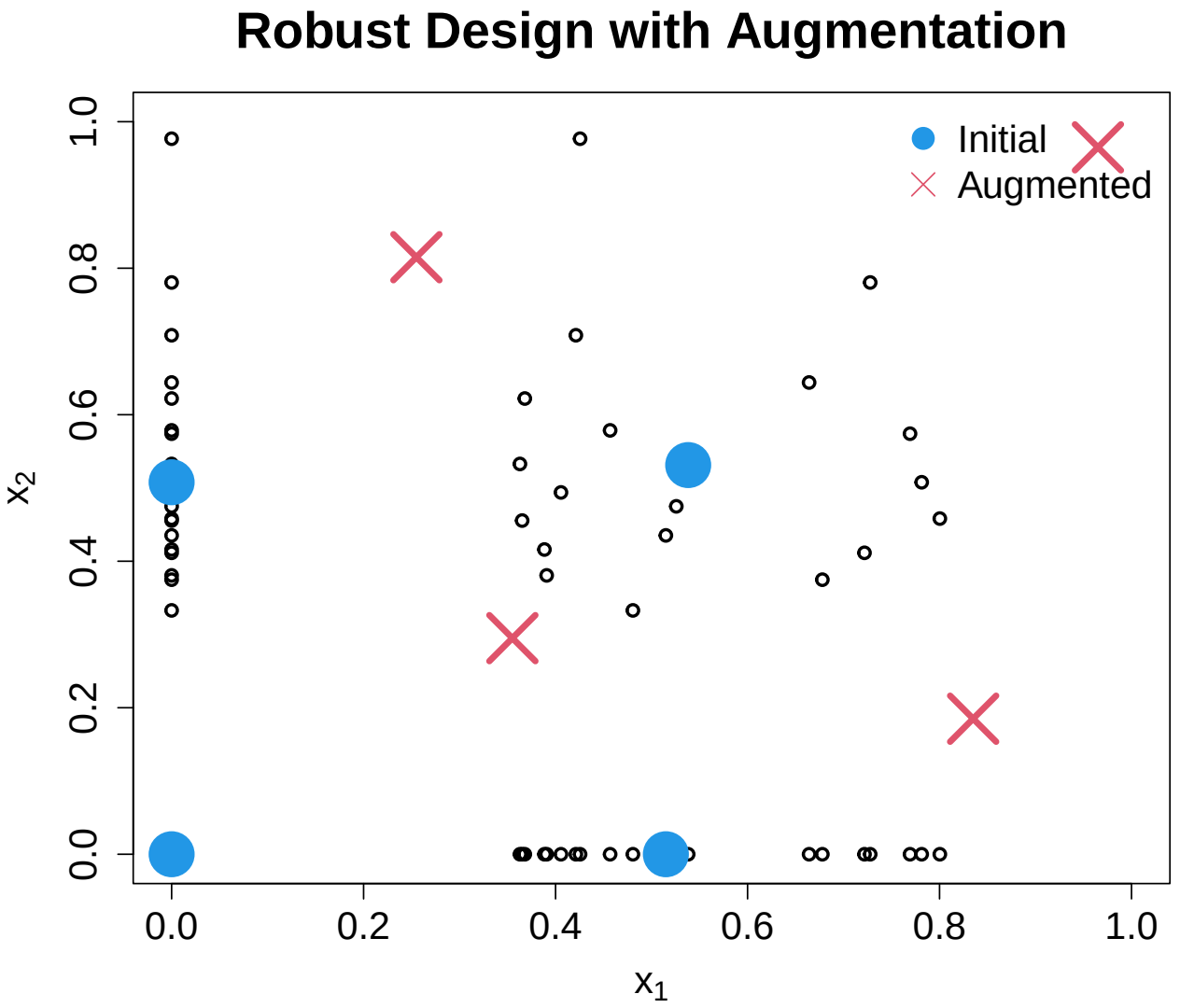
```
samp[, 1] <- 1 + 2 * samp[, 1]
samp[, 2] <- 2 + 1/2 * samp[, 2]

D <- NULL
for (i in 1:20) {
  D <- rbind(D, matrix(c(0, 0, 1/samp[i, 1], 0, 0, 1/samp[i, 2],
                         1/samp[i, 1], 1/samp[i, 2]), ncol = 2, byrow = TRUE))
}
ind <- sp.sample(D, 4, prob = rep(0.25/20, 80))

cand <- MaxProLHD(100, 2)$Design
a <- MaxProAugment(ExistDesign = D[ind, ], CandDesign = cand, nNew = 4)
Dnew <- a$Design[5:8, ]

plot(D, xlim = c(0, 1), ylim = c(0, 1), main = "Robust Design with Augmentation",
```

```
     cex.main = 2, xlab = expression(x[1]), ylab = expression(x[2]),
     cex.lab = 1.5, cex.axis = 1.5, lwd = 2)
points(D[ind, ], pch = 16, col = 4, cex = 4)
points(Dnew, col = 2, pch = 4, cex = 4, lwd = 4)
legend("topright", legend = c("Initial", "Augmented"),
       pch = c(16, 4), col = c(4, 2), cex = 1.5, bty = "n", pt.cex = 2)
```



4 Summary

This chapter covered:

1. **Locally D-optimal designs** for nonlinear models
2. **Pseudo-Bayesian D-optimal designs** averaging over parameter uncertainty
3. **Robust designs** accounting for parameter uncertainties using support points
4. **Design augmentation** for model uncertainties using MaxPro
5. **Functional output modeling** - Kriging for computationally expensive models with time-series output